오즈코딩스쿨 × 고용노동부 · 1인 창업가 개발부트캠프 7기

JavaScript 기초

DAY 12

반복문과 함수

같은 일을 여러 번 하는 반복문과
코드를 묶어 이름을 붙이는 함수를 배웁니다.

강사 김태민

TODAY'S GOAL

오늘 끝나면 할 수 있는 것

1

for · while로 같은 일을 반복할 수 있다

반복 횟수를 정하는 세 부분(초기화 · 조건 · 증감)을 읽고, break · continue로 흐름을 바꿉니다.

2

함수를 만들고 불러 쓸 수 있다

함수 선언식 · 함수 표현식 · 화살표 함수를 구분하고, 매개변수로 값을 받습니다.

3

return으로 결과를 돌려받을 수 있다

함수 안에서 계산한 값을 밖으로 꺼내 변수에 담고 다시 씁니다.

오늘의 결과물

2단부터 9단까지 출력하는 gugudan.js와 합계 · 평균을 돌려주는 함수가 담긴 sum.js

ROADMAP

오늘의 흐름

01

for 반복문

세 부분과 실행 순서
중첩 for

02

while과 흐름 바꾸기

break · continue
실습 01

실습 포함

03

함수 만들기

선언식 · 표현식
화살표 함수

04

매개변수와 return

값을 받고 돌려주기
실습 02

실습 포함

앞의 두 섹션은 같은 일을 되풀이하는 반복문, 뒤의 두 섹션은 코드를 묶어 다시 쓰는 함수입니다.
오늘도 화면은 바꾸지 않고 모든 결과를 Console에서 봅니다.

되짚기 · 01

어제 배운 것 — 조건으로 갈 길 나누기

```
// DAY 11에서 한 것
let total = 0;
total += 12000;              // += : 값을 쌓기
total += 3000;

const num = 7;
if (num % 2 === 0) {         // % : 짝수 판별
  console.log("짝수");
} else {
  console.log("홀수");
}
```

오늘은 되풀이

같은 계산을 여러 번
되풀이하는 반복문이
나옵니다.

그리고 묶기

자주 쓰는 코드를 함수로
묶어 이름을 붙이고
부를 때마다 실행합니다.

되짚기 · 02

같은 줄을 아홉 번 쓰는 구구단 3단

```
// 구구단 3단을 이렇게 쓰면
console.log("3 * 1 = 3");
console.log("3 * 2 = 6");
console.log("3 * 3 = 9");
// … 여섯 줄 더
console.log("3 * 9 = 27");

// 9999까지 출력하려면?
```

반복문으로 한 번에

바뀌는 숫자만 변수로 두고
같은 줄을 되풀이하게
합니다.

함수로 다시 쓰기

3단 코드를 다른 곳에서도
쓰려면 함수로 묶어
이름으로 부릅니다.

준비

오늘의 셋팅

```
Day12/                    ← 같은 저장소 안에 만듭니다
  gugudan.html            ← gugudan.js 를 연결합니다 (실습 01)
  gugudan.js
  sum.html                ← sum.js 를 연결합니다 (실습 02)
  sum.js
$ git status
/* 오늘도 main에 바로 커밋합니다 */
```

여러 줄은 Shift + Enter

Console에서 for 문처럼
여러 줄을 칠 때는
Shift + Enter로 줄바꿈.

실습은 파일로

구구단과 합계 함수는
.js 파일에 쓰고
새로고침해 확인합니다.

01

for 반복문

반복문이란 · for의 세 부분 · 실행 순서 · 합계 쌓기 · 중첩 for

조건이 맞는 동안 같은 일 되풀이



접시가 남아 있는 동안 같은 동작을 되풀이하는 설거지

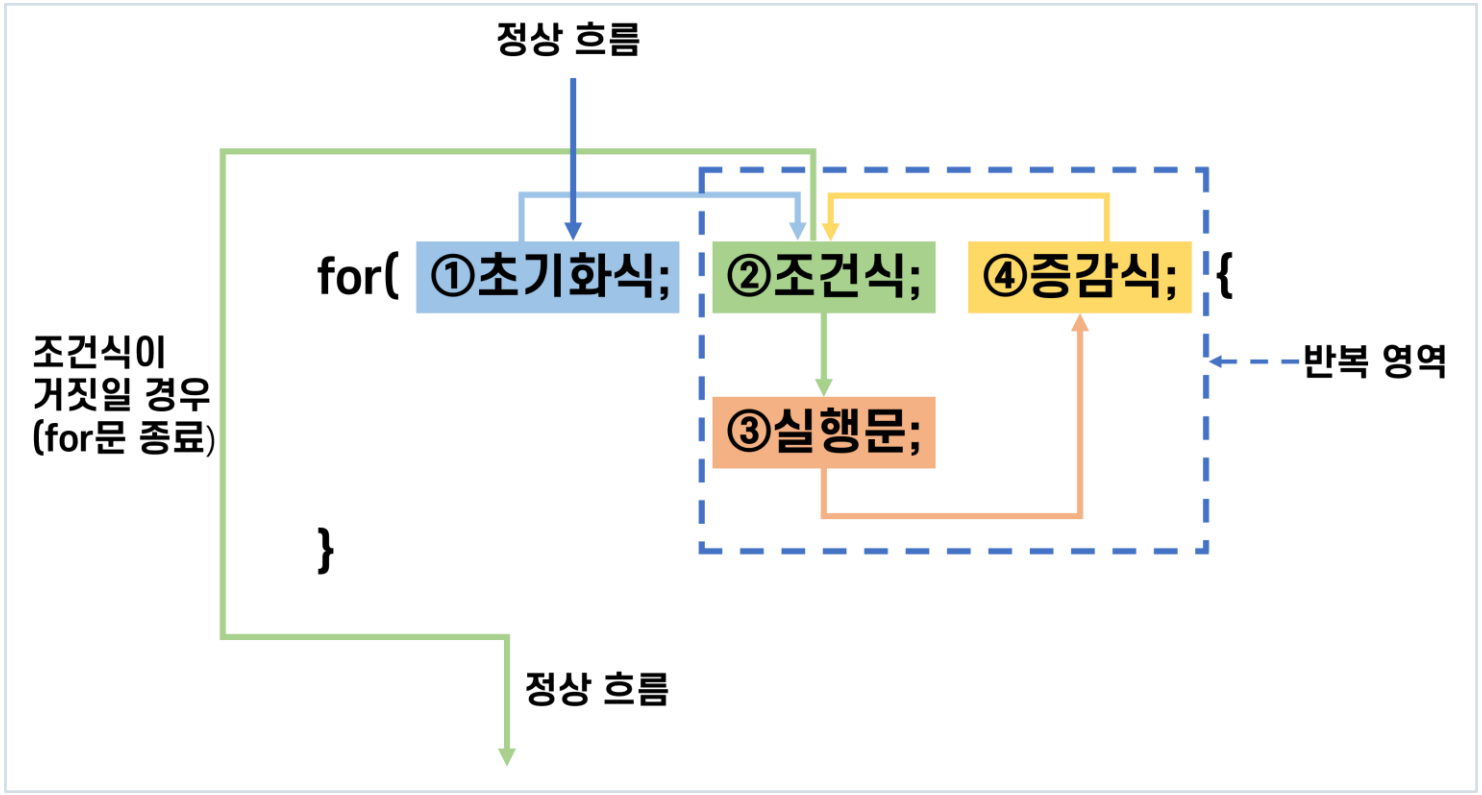
조건 + 되풀이

「접시가 남아 있는 동안」
이 조건이고, 닦기가
되풀이할 일입니다.

오늘은 세 가지

while, do...while, for.
이 밖에 for...of ·
for...in도 있습니다.

세 부분으로 횟수를 정하는 for



①초기화식 → ②조건식 → ③실행문 → ④증감식 → 다시 ②

초기화식은 한 번

let i = 0 처럼 반복에
쓸 변수를 처음 한 번만
만듭니다.

조건이 거짓이면 끝

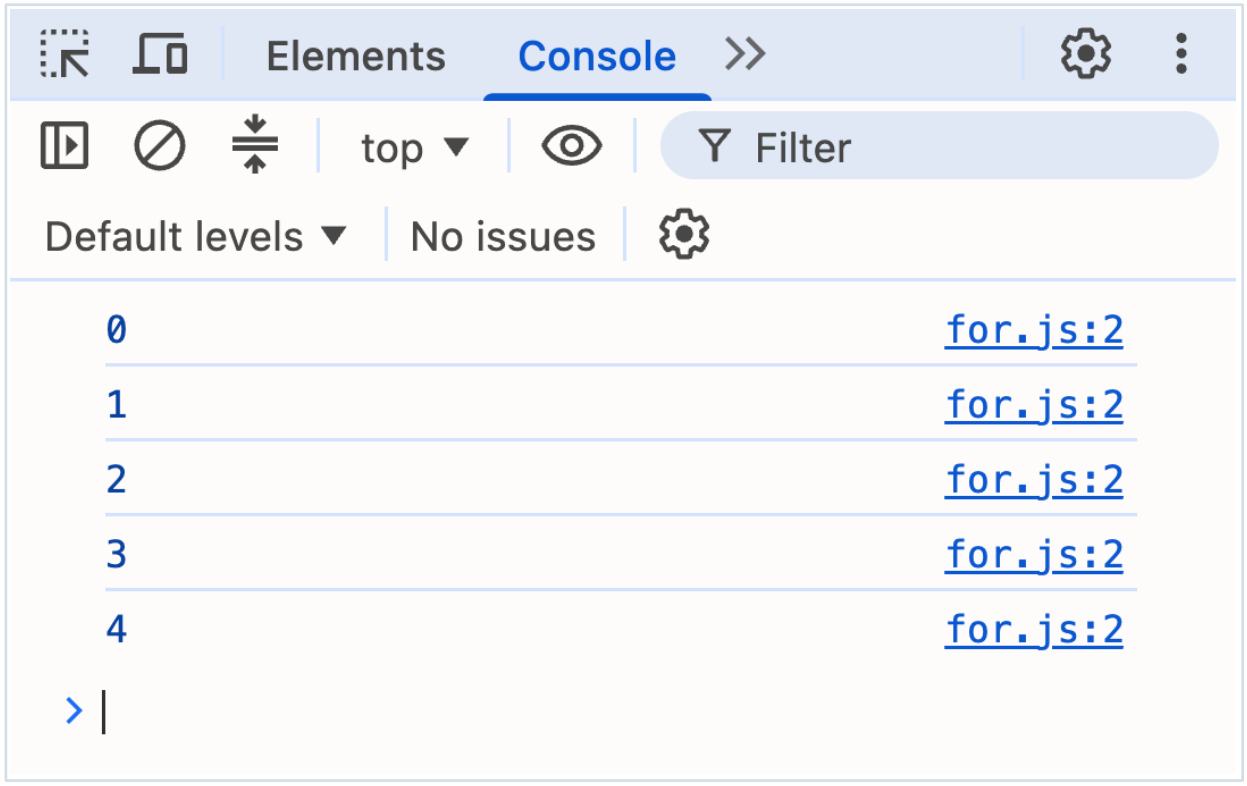
조건식 → 실행문 → 증감식을
되풀이하다가 조건이
false가 되면 빠져나옵니다.

0부터 4까지 다섯 번 실행

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}
```

i가 바뀌는 순서

0 → 1 → 2 → 3 → 4 : 조건이 참이라 출력
5 : i < 5가 false — 반복을 끝냅니다
i++ 는 i = i + 1 과 같습니다



for가 도는 순서

1	① 초기화	let i = 0 — 반복을 시작할 때 한 번만 실행
2	② 조건	i < 5 인지 검사 — 참이면 ③으로, 거짓이면 반복 끝
3	③ 실행	{ } 안의 코드를 실행 — console.log(i)
4	④ 증감	i++ 로 i를 1 늘림
5	다시 ②	조건이 거짓이 될 때까지 ② → ③ → ④ 를 되풀이

증감식이 없거나 조건이 늘 참이면 끝나지 않습니다 — 섹션 02에서 봅니다

01 · 합계

반복하며 += 로 쌓기

```
let total = 0;                  // 반복 밖에서 0으로 시작
for (let i = 1; i <= 10; i++) {
  total += i;                   // 1, 3, 6, 10, … 차례로 쌓임
}
console.log(total);             // 55
```

0에서 시작

합계를 담을 변수는 반복문 밖에서 0으로 만듭니다.

<= 로 10까지

i <= 10 이라서 10까지 더합니다. i < 10 이면 9까지라 45가 나옵니다.

바뀌는 숫자만 변수로 두기

```
for (let i = 1; i <= 9; i++) {
  console.log(`3 * ${i} = ${3 * i}`);
}
```

아홉 줄이 세 줄로

곱하는 수 1~9만 i로 바꾸고,
나머지 글자는 그대로 둡니다.

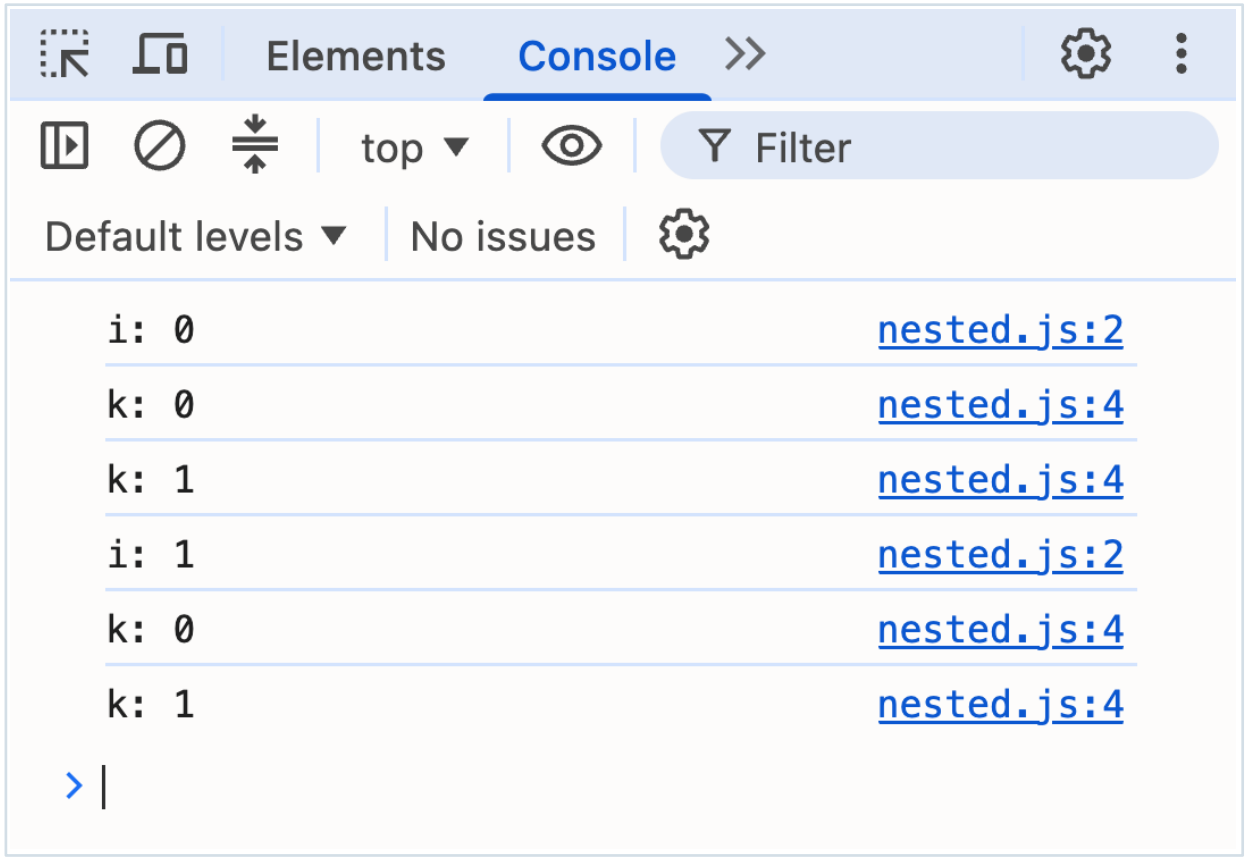


for 안에 for — 안쪽이 다 돈 뒤 바깥이 한 번

```
for (let i = 0; i < 2; i++) {
  console.log(`i: ${i}`);
  for (let k = 0; k < 2; k++) {
    console.log(`k: ${k}`);
  }
}
```

읽는 법

i가 0일 때 k가 0, 1로 두 번,
i가 1일 때 다시 k가 0, 1로 두 번.



여러 값을 하나씩 꺼낼 때도 for

```
const fruits = ["바나나", "사과", "오렌지"];

for (let i = 0; i < fruits.length; i++) {    // length : 3
  console.log(fruits[i]);                     // 0번째부터
}
// 바나나
// 사과
// 오렌지
```

length는 개수

값이 몇 개든
i < fruits.length 로
끝까지 돕니다.

번호는 0부터

fruits[0]이 "바나나".
배열은 DAY 13에서
자세히 배웁니다.

여기까지 정리

1	**for**	for (초기화; 조건; 증감) — 횟수가 정해진 반복에
2	**순서**	초기화 한 번, 그 뒤 조건 → 실행 → 증감을 되풀이
3	**합계**	반복 밖에서 0으로 시작하고, 안에서 += 로 쌓기
4	**중첩**	안쪽 for가 다 돈 뒤 바깥 for가 한 번 돈다

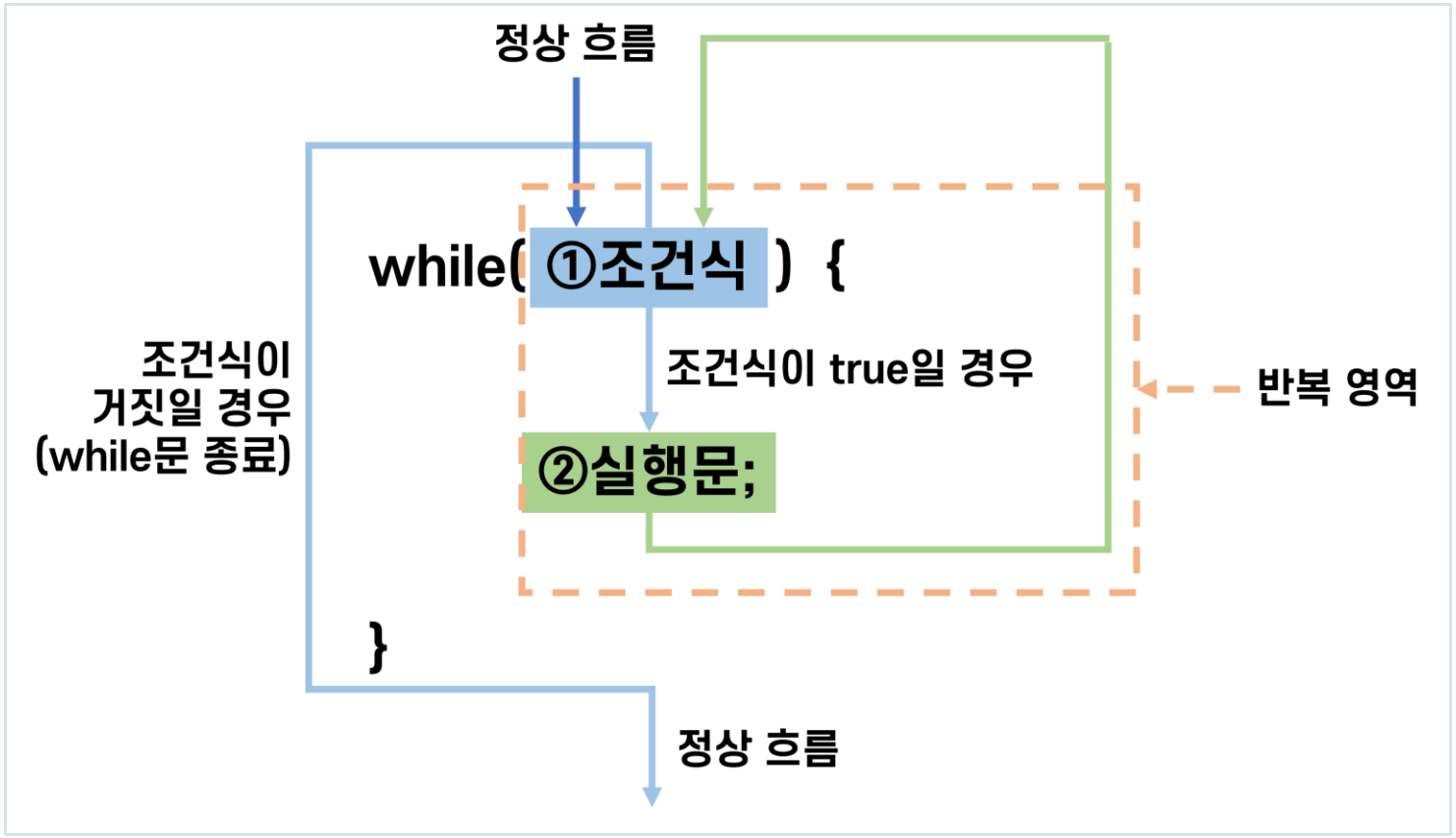
다음은 조건만 보고 되풀이하는 while입니다

02

while과 흐름 바꾸기

while · 끝나지 않는 반복 · do...while · break · continue · 실습 01

while — 조건이 참인 동안 되풀이



①조건식이 참이면 ②실행문, 다시 ①로 — 거짓이면 while을 빠져나감

조건만 있습니다

for와 달리 초기화와
증감은 while 바깥과
안쪽에 따로 씁니다.

횟수를 모를 때

「돈이 남아 있는 동안」
처럼 몇 번 돌지
미리 모를 때 씁니다.

02 · while

조건을 바꾸는 줄은 필수

```
let num = 1;
while (num <= 5) {
  console.log(num);
  num++;                    // 이 줄이 없으면 끝나지 않습니다
}
// 1 2 3 4 5

while (true) { … }          // 조건이 늘 참 — 끝나지 않음
```

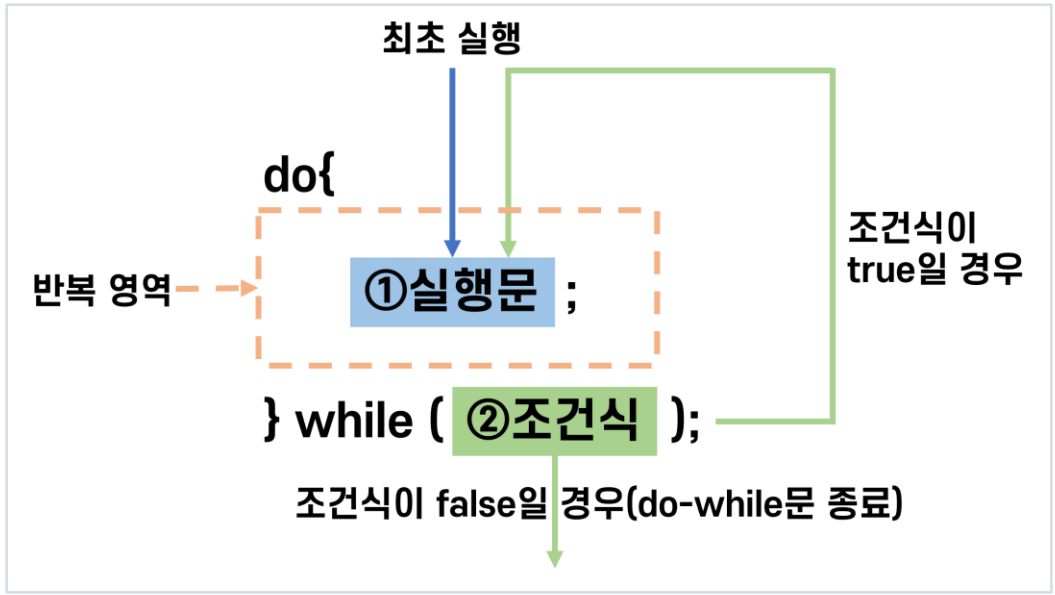
끝나지 않는 반복

조건이 계속 참이면
반복이 끝나지 않고
페이지가 멈춥니다.

멈췄다면

그 탭을 닫고
코드를 고친 뒤
다시 엽니다.

do...while — 한 번은 먼저 실행



①실행문을 먼저, 그다음 ②조건식 — 참이면 다시 ①로

```
do {
  console.log("조건이 false여도 한 번은 실행");
} while (false);
```

실행이 먼저

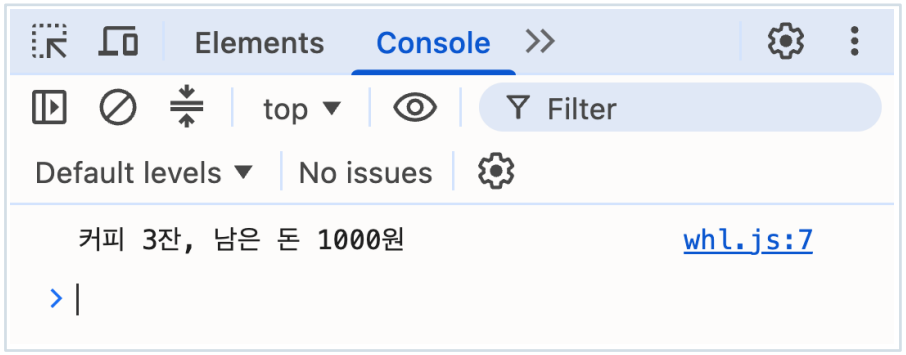
조건을 나중에 보므로
적어도 한 번은
실행됩니다.

끝에 ;

while ( ) 뒤에
세미콜론을 붙입니다.
while 문과 다른 점.

몇 번 돌지 모를 때는 while

```
let money = 10000;
let cups = 0;
while (money >= 3000) {       // 3000원 이상 남은 동안
  money -= 3000;
  cups++;
}
console.log(`커피 ${cups}잔, 남은 돈 ${money}원`);
```



조건만 압니다

몇 잔인지는 모르고
「3000원 이상 남은 동안」
이라는 조건만 압니다.

안쪽에서 바꾸기

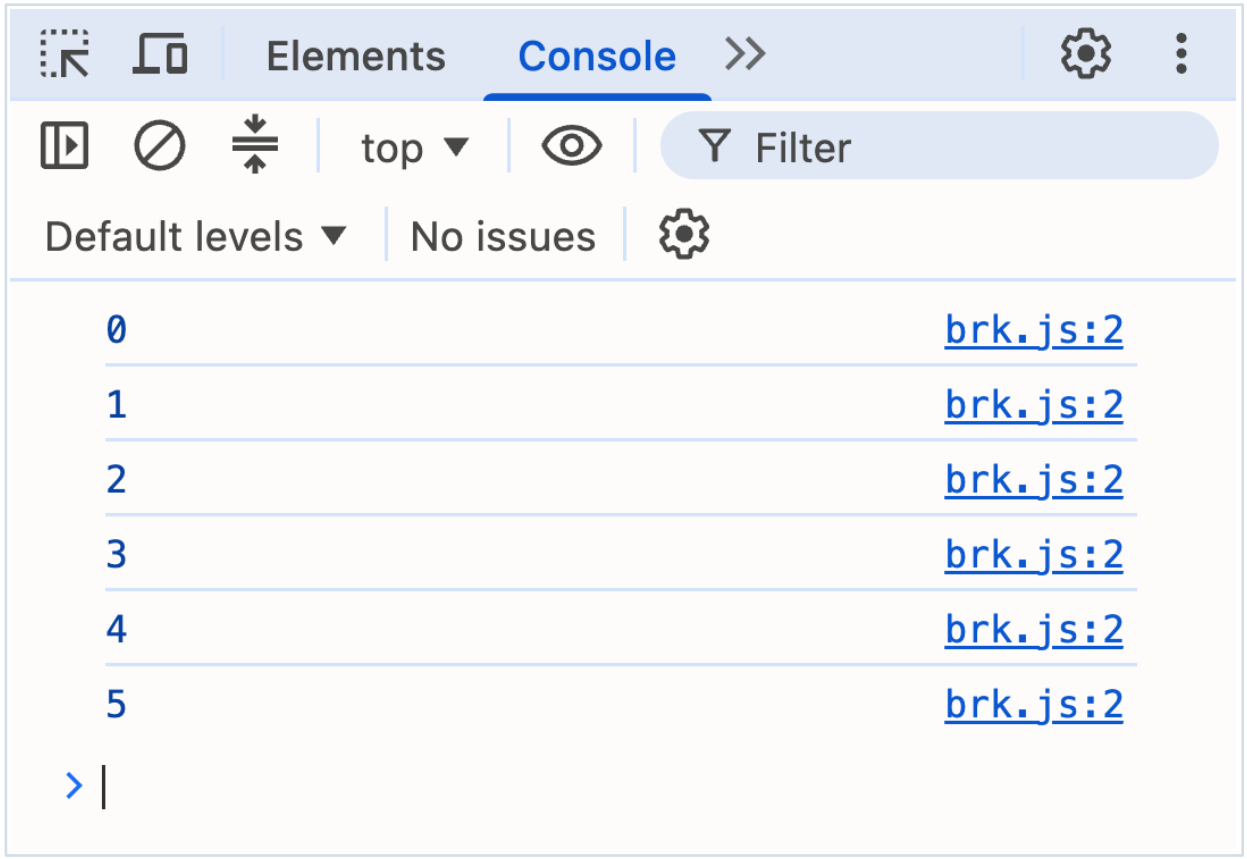
money -= 3000 이
조건을 바꾸는 줄이라
반복이 끝납니다.

break — 반복을 바로 끝내기

```
for (let i = 0; i < 10; i++) {
  console.log(i);
  if (i === 5) {
    break;
  }
}
```

찾으면 멈추기

원하는 값을 찾았을 때
나머지 반복을 하지 않습니다.

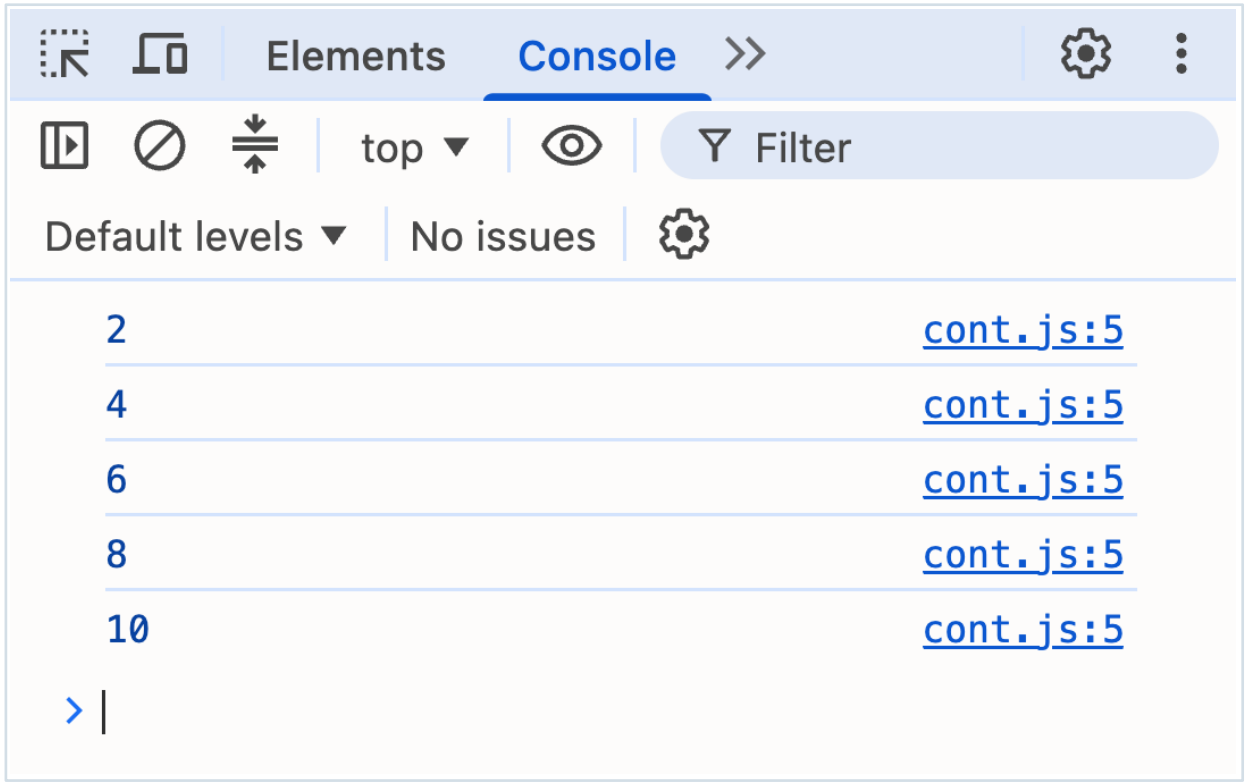


continue — 이번 차례만 건너뛰기

```
for (let i = 1; i <= 10; i++) {
  if (i % 2 === 1) {
    continue;
  }
  console.log(i);
}
```

break와 다른 점

break는 반복을 끝내고,
continue는 다음 차례로 넘어갑니다.



02 · 고르는 법

횟수를 알면 for, 조건만 알면 while

횟수를 알 때

for

1부터 9까지, 배열의
처음부터 끝까지처럼
몇 번 돌지 알 때.

조건만 알 때

while

돈이 남아 있는 동안처럼
몇 번 돌지는 모르고
조건만 알 때.

한 번은 먼저

do…while

조건과 상관없이
적어도 한 번은 실행해야
할 때.

break는 반복을 끝내고, continue는 이번 차례만 건너뜁니다

실습 01

구구단 출력하기

gugudan.js에 3단, 2단부터 9단까지, 홀수 단만, 50을 넘는 첫 곱 찾기를 차례로 만듭니다.
for · 중첩 for · continue · break를 한 파일에서 모두 씁니다.

```
// 2단부터 9단까지 — 바깥은 단, 안쪽은 곱하는 수
for (let dan = 2; dan <= 9; dan++) {
  let line = `${dan}단 :`;
  for (let i = 1; i <= 9; i++) {
    line += ` ${dan * i}`;
  }
  console.log(line);
}

// 홀수 단만 — 짝수 단은 continue
// 7단에서 처음으로 50을 넘는 곱 — 찾으면 break
```

```
2단 : 2 4 6 8 10 12 14 16 18                gugudan.js:14
3단 : 3 6 9 12 15 18 21 24 27               gugudan.js:14
4단 : 4 8 12 16 20 24 28 32 36              gugudan.js:14
5단 : 5 10 15 20 25 30 35 40 45             gugudan.js:14
6단 : 6 12 18 24 30 36 42 48 54             gugudan.js:14
7단 : 7 14 21 28 35 42 49 56 63             gugudan.js:14
8단 : 8 16 24 32 40 48 56 64 72             gugudan.js:14
9단 : 9 18 27 36 45 54 63 72 81             gugudan.js:14
3단은 홀수 단입니다                          gugudan.js:22
5단은 홀수 단입니다                          gugudan.js:22
7단은 홀수 단입니다                          gugudan.js:22
9단은 홀수 단입니다                          gugudan.js:22
7 * 8 = 56 에서 처음으로 50을 넘었습니다      gugudan.js:28
>
```

완성 파일의 실행 결과 (2단부터 끝까지)

03

함수 만들기

함수란 · 선언과 호출 · 함수 표현식 · 화살표 함수 · 선언 전에 부르면

코드를 묶어 이름을 붙인 함수

```
function gugudan() {           // 함수를 만든다 = 정의
  for (let i = 1; i <= 9; i++) {
    console.log(`3 * ${i} = ${3 * i}`);
  }
}

gugudan();                     // 함수를 부른다 = 호출
gugudan();                     // 부를 때마다 다시 실행
```

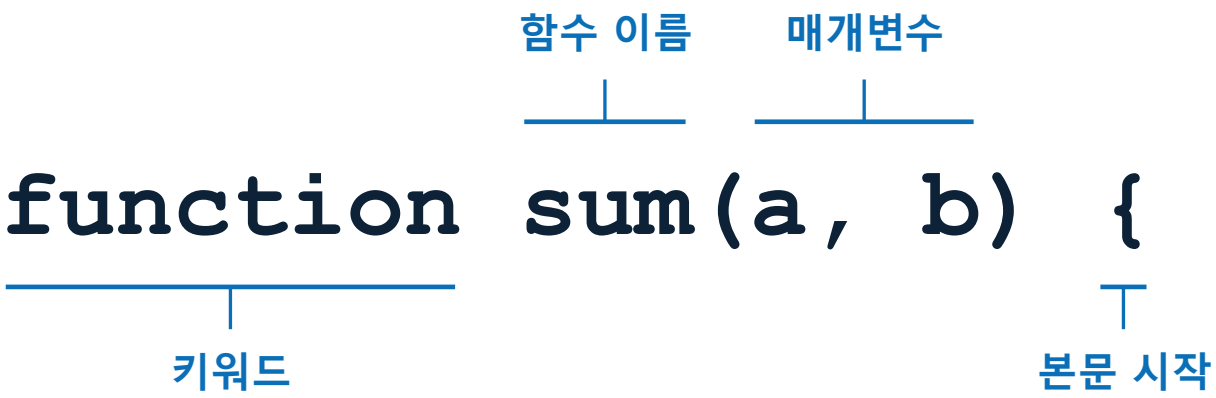
정의만 하면 그대로

function으로 만들기만 하면
아무 일도 일어나지
않습니다.

() 를 붙여 호출

gugudan(); 처럼 이름 뒤에
괄호를 붙여야 그 안의
코드가 실행됩니다.

한 줄씩 나눠 읽기



function 키워드 · 함수 이름 · (매개변수) · { 본문 }

부를 때는 sum(10, 20); 처럼 함수 이름 뒤 괄호 안에 값을 넣습니다.

함수를 변수에 담기

```
const gugudan = function () {
  for (let i = 1; i <= 9; i++) {
    console.log(`3 * ${i} = ${3 * i}`);
  }
};                              // 변수에 담는 문장이라 ;

gugudan();
```

함수도 값입니다

숫자나 문자열처럼
변수에 담을 수 있습니다.
typeof는 "function".

부르는 법은 같습니다

담은 변수 이름 뒤에
괄호를 붙여
gugudan(); 으로 부릅니다.

=> 로 더 짧게 쓰는 화살표 함수

```
const gugudan = () => {
  for (let i = 1; i <= 9; i++) {
    console.log(`3 * ${i} = ${3 * i}`);
  }
};

gugudan();
```

function 대신 =>

( ) => { } 모양입니다.
괄호 안이 매개변수,
{ } 안이 본문입니다.

변수에 담아 씁니다

함수 표현식처럼
const 변수에 담고
이름으로 부릅니다.

03 · 비교

세 가지 모양, 차이는 부를 수 있는 때

	모양	만들기 전 줄에서 부르면
함수 선언식	`function gugudan() { … }`	불러집니다
함수 표현식	`const gugudan = function () { … };`	ReferenceError
화살표 함수	`const gugudan = () => { … };`	ReferenceError

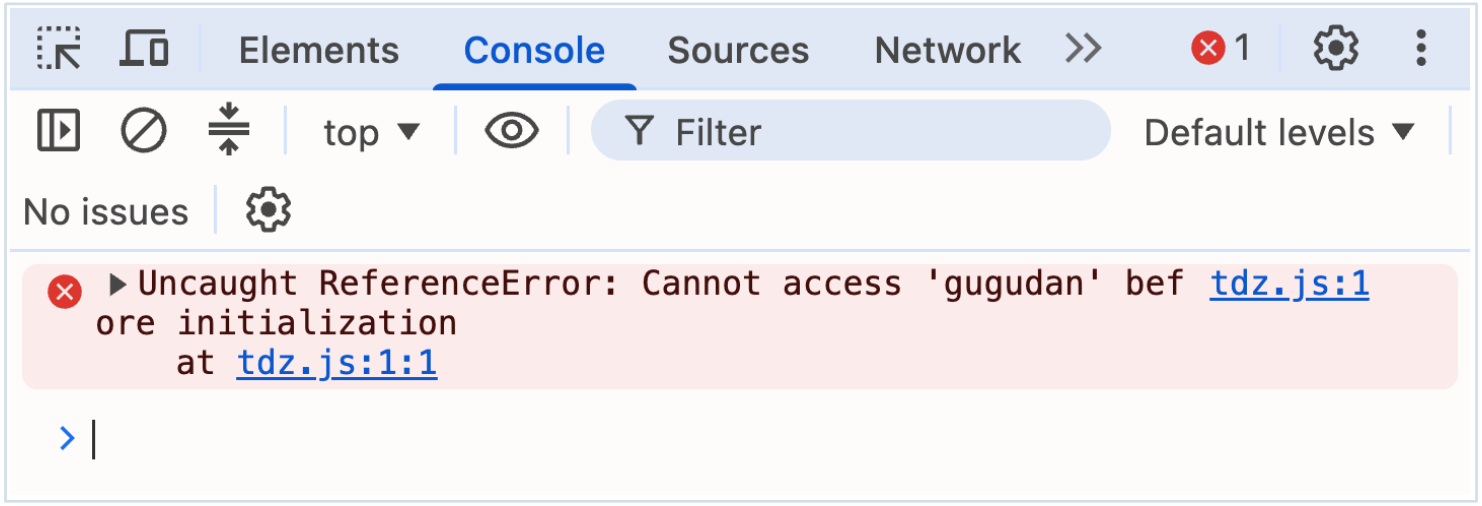
이 과정에서는 먼저 만들고, 그 아래에서 부릅니다

셋 중 무엇으로 만들든 부르는 법(이름 뒤에 괄호)은 같습니다.

만들기 전에 부르면 ReferenceError

```
gugudan();                    // 아직 만들기 전

const gugudan = () => {
  console.log("3단");
};
```



읽는 법

Cannot access 'gugudan' before initialization — 만들기 전에는 못 씁니다.

고치는 법

함수를 만드는 줄을 부르는 줄보다 위로 옮깁니다.

여기까지 정리

1	**함수**	코드를 묶어 이름을 붙인 것 — 정의하고, 이름() 으로 호출
2	**선언식**	function 이름() { } — 만들기 전 줄에서도 부를 수 있음
3	**표현식**	const 이름 = function () { }; — 변수에 담은 함수
4	**화살표**	const 이름 = () => { }; — ES2015에 추가된 짧은 모양

다음은 함수에 값을 넣고, 결과를 돌려받는 법입니다

04

매개변수와 return

매개변수와 인수 · 기본값 · return · 화살표 함수의 짧은 return · 실습 02

5단, 8단마다 따로 만든 함수

```
function gugudan5() {
  for (let i = 1; i <= 9; i++) {
    console.log(`5 * ${i} = ${5 * i}`);
  }
}
```

```
function gugudan8() {
  for (let i = 1; i <= 9; i++) {
    console.log(`8 * ${i} = ${8 * i}`);
  }
}
```

다른 것은 단 수 하나뿐 — 그 자리를 밖에서 받아 오면 됩니다

함수가 밖에서 값을 받는 자리를 매개변수, 부를 때 넣는 값을 인수라고 합니다.

매개변수는 받는 자리, 인수는 넣는 값

```
function gugudan(dan) {            // dan : 매개변수
  for (let i = 1; i <= 9; i++) {
    console.log(`${dan} * ${i} = ${dan * i}`);
  }
}

gugudan(3);                        // 3 : 인수 → 3단
gugudan(5);                        // 5 : 인수 → 5단
```

매개변수

함수를 만들 때 괄호 안에
적는 이름입니다.
받은 값이 담깁니다.

인수

함수를 부를 때 괄호 안에
넣는 값입니다. 부를 때마다
다른 값을 넣습니다.

여러 개면 순서대로 짝짓기

```
function introduce(name, age) {
  console.log(name, age);
}

introduce("김오즈", 28);          // 김오즈 28
introduce("김오즈");              // 김오즈 undefined ← 모자람
introduce("김오즈", 28, "서울"); // 김오즈 28 ← "서울"은 받는 자리 없음
```

순서대로

이름이 아니라 순서로
짝을 짓습니다.
순서를 바꾸면 값도 바뀜.

모자라면 undefined

인수를 빠뜨려도 오류는
나지 않고 undefined가
담깁니다.

인수를 빠뜨리면 쓸 기본값

```
function greet(name = "회원") {
  console.log(`${name}님, 안녕하세요`);
}

greet("김오즈");      // 김오즈님, 안녕하세요
greet();              // 회원님, 안녕하세요 ← 기본값
```

= 로 적습니다

매개변수 이름 뒤에
= 기본값 을 붙입니다.

넣으면 넣은 값

인수를 넣으면 기본값은
쓰이지 않습니다.

return — 결과를 밖으로 돌려주기

```
function sum(num1, num2) {
  const result = num1 + num2;
  return result;              // 부른 곳으로 돌려줌
}

const answer = sum(10, 20);   // 돌려받은 30을 담음
console.log(answer);          // 30

console.log(result);          // ReferenceError: result is not defined
```

반환값

return 뒤의 값이
함수를 부른 자리로
돌아갑니다.

안의 변수는 안에서만

함수 안의 result는
밖에서 보이지 않습니다.
꺼내려면 return.

return을 만나면 함수 끝

```
function sum(num1, num2) {
  if (typeof num1 !== "number" || typeof num2 !== "number") {
    return;                    // 숫자가 아니면 여기서 끝
  }
  return num1 + num2;
}

console.log(sum(10, 20));      // 30
console.log(sum("a", "b"));    // undefined
```

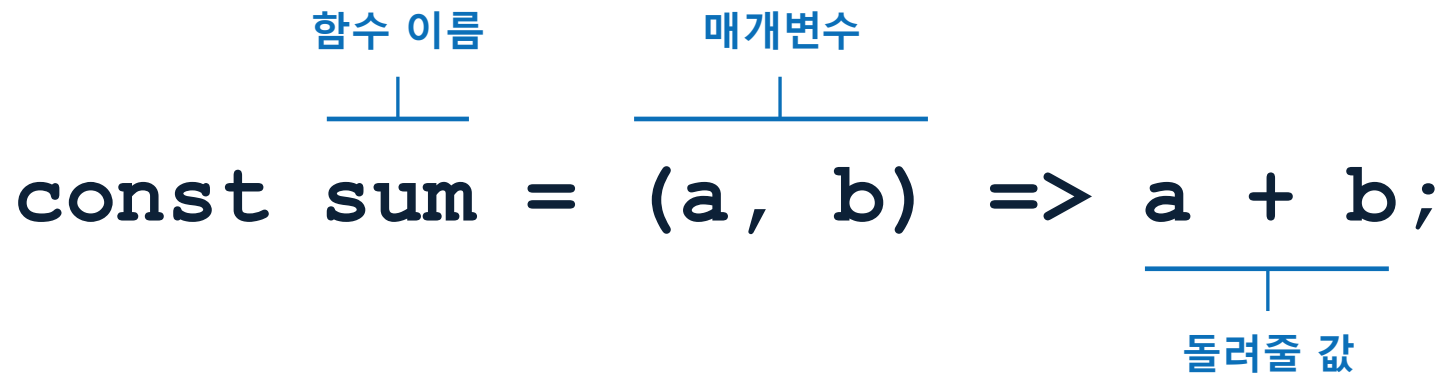
아래 줄은 건너뜀

return 다음 줄이 있어도
실행하지 않고 함수를
끝냅니다.

없으면 undefined

return이 없거나 값 없이
return만 쓰면
undefined가 돌아옵니다.

한 줄 계산은 { } 와 return 생략



const sum = (a, b) => { return a + b; }; 와 같습니다

{ } 를 쓰면 return을 직접 적어야 합니다. { } 안에 return이 없으면 undefined가 돌아옵니다.

실습 02

합계 함수 만들기

sum.js에 합계 · 구구단 · 점수 합계 · 조건에 맞는 개수를 돌려주는 함수를 차례로 만듭니다.
함수 선언식 · 표현식 · 화살표 함수를 한 번씩 쓰고, 결과는 return으로 돌려받습니다.

만들 함수

① sumTo(n) — 1부터 n까지의 합을 return (함수 선언식)

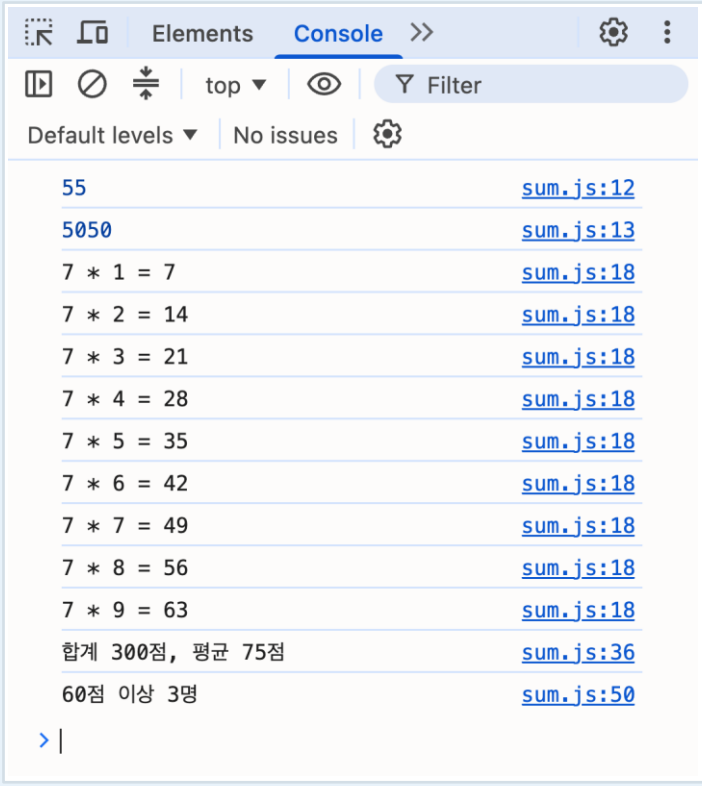
② printDan(dan) — 그 단을 출력 (함수 표현식)

③ sumScores(list) — 점수 목록의 합계를 return (화살표 함수)

④ countPass(list) — 60점 이상인 개수 (60점 미만은 continue)

⑤ 합계와 평균, 60점 이상 개수를 출력

다 되면 커밋합니다 — 이어서 일부러 틀려 봅니다



실습 02

일부러 틀려 보기

완성한 sum.js를 한 곳씩 바꿔 저장하고, 결과가 어떻게 바뀌는지 본 뒤 되돌립니다.
세 번째만 빨간 줄이 뜹니다. 나머지 둘은 결과를 읽고 원인을 찾습니다.

바꿀 곳	결과
① sumTo의 return total; 지우기	Console에 55 · 5050 대신 undefined 두 줄
② sumTo의 i <= n 을 i < n 으로	55 대신 45, 5050 대신 4950 — 하나씩 모자람
③ sumScores를 만들기 전 줄에서 부르기	ReferenceError: Cannot access 'sumScores' before initialization

세 가지를 다 보고 되돌렸으면 커밋합니다 — 실습 02 끝

점검

반복문과 함수에서 자주 나오는 세 가지

!

undefined가 나와요

함수에 return이 없거나
{ } 화살표 함수에서
return을 빠뜨렸습니다.

!

하나씩 모자라요

<= 자리에 < 를 썼습니다.
끝 값까지 넣으려면
<= 입니다.

!

페이지가 멈췄어요

증감식이나 조건을 바꾸는
줄이 없어 반복이
끝나지 않습니다.

반복문과 함수는 끝 값과 돌려주는 값을 확인합니다

첫 값과 마지막 값을 console.log로 찍어 보고, 함수의 결과는 변수에 담아 찍어 봅니다.

WRAP UP

오늘 정리

개념

for는 초기화 한 번, 조건 → 실행 → 증감 되풀이

while은 조건만 보고 되풀이한다

break는 반복을 끝내고, continue는 건너뛴다

함수는 코드를 묶어 이름을 붙인 것

매개변수는 받는 자리, 인수는 넣는 값

return은 결과를 돌려주고 함수를 끝낸다

직접 쓸 수 있어야 하는 것

for로 1부터 n까지 합 구하기

중첩 for로 구구단 출력하기

while로 조건이 맞는 동안 반복하기

세 가지 모양으로 함수 만들기

매개변수로 값을 받아 쓰기

return으로 결과를 돌려받기

참고교재

교재에서 함께 읽을 곳

오늘 수업과 이어지는 장입니다. 복습할 때 장 번호로 찾아 읽습니다.

길벗캠퍼스

『백엔드 입문자를 위한 모던 자바스크립트 & Node.js』

이창현 지음

6장. 합리적인 사고, 반복문
7장. 필요할 때 호출, 함수

위키북스

『모던 자바스크립트 Deep Dive』

이웅모 지음

8장. 제어문
12장. 함수

NEXT

DAY 13

배열과 객체 기초

여러 값을 순서대로 담는 배열과
이름을 붙여 담는 객체를 배우고
실제 API 데이터 구조를 읽을 준비를 합니다.

수고하셨습니다.